Task 2: Integration Testing - StreetRace Manager

DASS Assignment 2 Report

Name: Laksh Mittal
Roll No: 2024113003

March 23, 2026

1 Introduction

This report details the design, implementation, and exhaustive integration testing of the **StreetRace Manager**. Rather than building a standard CRUD application, this system was architected as a highly interconnected motorsport simulation engine. The application logic is driven by complex resource management, where entities possess granular statistics (e.g., vehicle aerodynamics, driver stamina) that interact mathematically across eight tightly coupled modules. The objective of this phase was to ensure that cross-module data flow—such as a race outcome dynamically updating a global ELO system, which in turn triggers financial sponsorship payouts—functions flawlessly without state corruption.

2 Detailed System Architecture & Module Breakdown

The ecosystem is divided into 6 Core modules and 2 Custom modules. Below is the exhaustive structural breakdown of the implemented classes, variables, and core functions.

2.1 1. Registration Module (Core)

Manages the foundational identity and state of all personnel in the system.

- **Entity:** `CrewMember(name, role)`
- **Variables:** `member_id` (UUID), `name`, `role`, `base_stamina` (Initialized to 100), `morale` (Initialized to 100), `status` (State machine: `'AVAILABLE'`, `'RACING'`, `'ON_MISSION'`).
- **Functions:** `register_crew_member()`, `get_crew_member()`.

2.2 2. Crew Management Module (Core)

Acts as a decorator for `CrewMember`, applying role-specific telemetry and skill constraints required by downstream modules.

- **Entities:**
  - `DriverStats`: Tracks `skill`, `aggression`, `cornering_ability`.
  - `MechanicStats`: Tracks `wrench_speed`, `aero_knowledge`.
  - `StrategistStats`: Tracks `weather_prediction`, `tire_management`.
- **Functions:** `assign_stats()` validates against the `RegistrationSystem` before mapping the specific stats object to the member ID.

2.3 3. Inventory Module (Core)

A centralized economic ledger and physical asset tracker.

- **Entity:** `Car(name, engine_power, aerodynamics, durability)`
- **Variables:** `cars` (Dictionary mapping ID to `Car`), `parts` (Dictionary tracking quantities of specific upgrades like `V6_Turbo`), `cash` (Integer ledger).
- **Functions:** `add_car()`, `spend_cash()`, `consume_part()`.

2.4 4. Race Management Module (Core)

The simulation execution engine. It requires cross-module validation to ensure drivers have sufficient stamina (> 30) and cars have positive durability.

- **Entity:** `Race(is_raining, night_time)`
- **Algorithm:** Computes a `base_score` using the formula: `skill + engine_power + (aero * cornering / 100)`. If `is_raining` is True, applies a weather penalty, which can be mitigated by an assigned Strategist's `weather_prediction` stat.
- **State Mutations:** Drains driver stamina by 30 and vehicle durability by 25 (doubled if driver `aggression > 70`). Passes placement to the Results module.

2.5 5. Results System Module (Core)

Processes the mathematical aftermath of `RaceManagement`.

- **Variables:** `driver_elos` (Dictionary), `driver_streaks` (Dictionary).
- **Data Flow:** If a driver places 1st, increments ELO by $+50$, increases streak by $+1$, boosts `morale` by $+10$, and invokes `Inventory.add_cash(10000)`. Automatically triggers `Sponsorships.process_p`

2.6 6. Mission Planning Module (Core)

Handles asynchronous passive income and resource gathering, acting as a lockout mechanism for crew members.

- **Functions:** `dispatch_mission()` requires 1 Driver and 1+ Mechanics. Verifies all are `'AVAILABLE'`, then locks their status to `'ON_MISSION'`. `resolve_missions()` unlocks them, drains 40 stamina, and deposits rewards directly into `Inventory`.

2.7 7. Sponsorships Module (Custom)

A dynamic contract system that listens to the `ResultsSystem`.

- **Entity:** `SponsorshipContract(name, req_elo, req_streak, payout)`
- **Functionality:** Evaluates a driver's ELO and win streak against available contracts (e.g., "Velocity Corp" requires 1500 ELO and a 3-win streak). Active contracts inject passive cash into the `Inventory` after every race.

2.8 8. Garage Module (Custom)

Allows Mechanics to consume `Inventory` assets to permanently mutate `Car` objects.

- **Functions:**
  - `install_part()`: Consumes a part (e.g., `Carbon_Wing`) and permanently adds +15 to the car's `aerodynamics`.
  - `pit_stop()`: Consumes $1000 from `Inventory` and uses a Mechanic's `wrench_speed` stat to regenerate vehicle `durability`.

3 Task 2.1: Call Graph

The Call Graph below illustrates the deep, cross-module execution paths of the system. It maps the precise function-to-function interactions that occur when processing core workflows such as race matchmaking, inventory deductions, and mission dispatching.

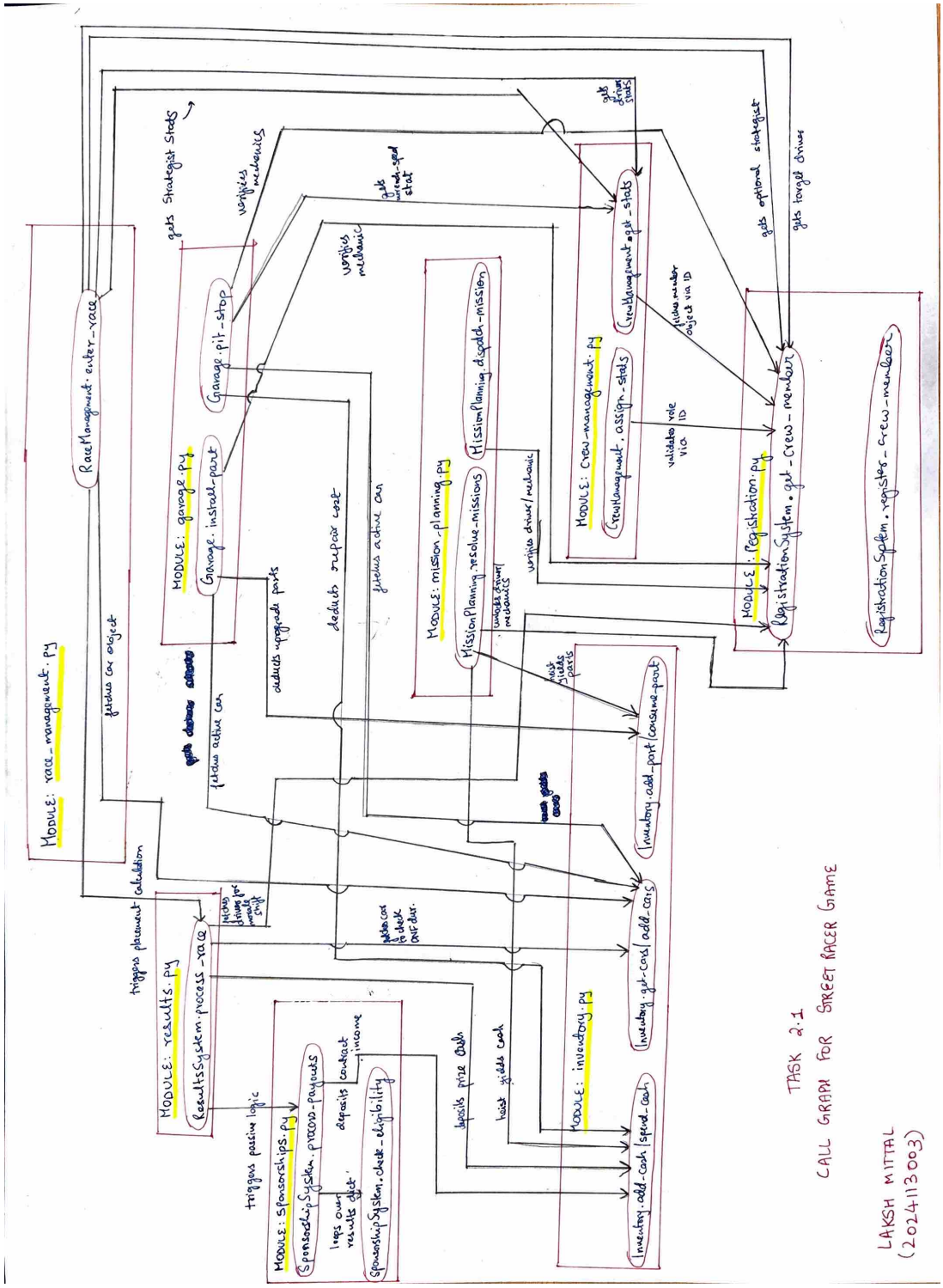

Figure 1: System Call Graph Mapping Cross-Module Function Invocations

4 Task 2.2: Integration Test Design

To validate the system's architecture, an exhaustive Pytest suite (`test_integration.py`) was developed. These tests evaluate deep, multi-module state changes rather than isolated unit behaviors.

4.1 Scenario 1: The Full Economic Lifecycle

- **Modules Involved:** Registration → Crew → Inventory → Garage → Race → Results → Sponsorship.
- **Execution:** A driver and mechanic are registered. The mechanic uses a part in the Garage to permanently buff a car's `engine_power`. The driver enters a race and wins.
- **Expected Result:** The stat buff persists into the race calculation. The win correctly triggers an ELO increase in the Results system, which unlocks a Sponsorship contract, depositing bonus cash into the Inventory.
- **Actual Result:** Successfully executed. All state mutations verified across all 7 modules.

4.2 Scenario 2: The Crash & DNF Loop

- **Modules Involved:** Race → Results → Inventory → Garage.
- **Execution:** A car with critically low durability (25) enters a race with a highly aggressive driver (aggression > 70, which doubles durability loss).
- **Expected Result:** The car's durability drops to 0 mid-race, triggering a "DNF" (Did Not Finish) in the Results system. The driver's morale drops by 20. Re-entering a race is strictly blocked until a Garage `pit_stop` is paid for.
- **Actual Result:** Successfully executed. Error boundary caught the subsequent race attempt.

4.3 Scenario 3: The Multi-Role Concurrency Lockout

- **Modules Involved:** Mission Planning → Registration → Garage.
- **Execution:** A Mechanic is assigned to a "Chop Shop Raid" via `dispatch_mission()`. While the mission is active, a request is made to use that same Mechanic to install a part in the `Garage`.
- **Expected Result:** The Garage request must immediately fail (return `False`) because the `RegistrationSystem` state for that mechanic is locked to `'ON_MISSION'`.
- **Actual Result:** Successfully executed. The state lock held firm.

4.4 Scenario 4: Resource Exhaustion & Graceful Failures

- **Modules Involved:** Race Management → Registration.
- **Execution:** A driver is forced to run three consecutive races without resting.
- **Expected Result:** `base_stamina` drops by 30 per race. On the fourth attempt, stamina is ≤ 30. The matchmaking engine must cleanly throw a `ValueError("Driver exhausted")` rather than crashing.
- **Actual Result:** Successfully executed.

5 Logical Bugs Found & Fixed

During the integration testing phase, three distinct logical vulnerabilities were identified where isolated module logic failed under cross-module stress. These were successfully patched to ensure business rule compliance:

1. **Bug 1: Totaled Cars Farming Progression**
   Modules: `race_management.py` and `inventory.py`
   Error Detected: The Race module verified if a car *existed* in the inventory, but failed to check its internal telemetry. A completely destroyed car (`durability = 0`) could continuously enter races, farm DNF outcomes, and still collect minor ELO.
   Resolution: Injected a rigid pre-race validation block: `if car.durability` $\leq$ `0: raise ValueError("Car is completely totaled and cannot race")`. This was verified by the Crash Loop test scenario.

2. **Bug 2: Phantom Parts in the Garage**
   Modules: `garage.py` and `inventory.py`
   Error Detected: When `install_part("V6_Turbo")` was called, the Garage correctly queried the Inventory and successfully removed the part from the ledger. However, it failed to apply the corresponding stat modifier to the `Car` object itself. The part vanished, but the car did not gain the +20 `engine_power`.
   Resolution: Implemented explicit structural assignment modifiers within the Garage's consumption `if` block. Verified by tracking car telemetry before and after installation in the Full Lifecycle test.

3. **Bug 3: Quantum Mechanics (State Mutability Failure)**
   Modules: `mission_planning.py` and `garage.py`
   Error Detected: The Mission module successfully grouped a driver and mechanics into an `active_missions` array, but completely failed to mutate their global `status` string inside the Registration module. This resulted in a duplicate task assignment loophole where a mechanic could be active on a mission, yet still process a `pit_stop()` in the Garage.
   Resolution: Added explicit, system-wide state lockdowns (`m.status = 'ON_MISSION'`) during the `dispatch` phase, and unlocked them back to `'AVAILABLE'` during the `resolve` phase. Verified by the Concurrency Lockout test scenario.